

Supplementary: Hyperparameter Configuration

To ensure a fair comparison among feature-selection (FS), feature-extraction (FE), and classification strategies, all experiments were conducted using a fixed set of predefined hyperparameters. The objective was not to optimize each individual model independently, but rather to evaluate the contribution of different pipeline components under a consistent experimental protocol. The complete pipeline configuration is summarized below.

S1. Data Partitioning and Class Rebalancing

The datasets were divided into training and testing subsets using a stratified 80/20 train-test split with a fixed random seed of 42 to preserve class distributions while ensuring reproducibility. Class imbalance was handled only within the training partition using Random Under-Sampling. For multiclass datasets, the target sampling size was defined according to the median class frequency, preventing excessive reduction toward the smallest class while reducing majority-class dominance.

Setting	Exact implementation
Global seed	42
NumPy seed	np.random.seed(42)
Train/test split	80% / 20%
Split stratification	stratify=y
Split seed	random_state=42
Outer CV	RepeatedStratifiedKFold
CV folds	5
CV repetitions	3
Total outer validation folds	15 per pipeline configuration
Primary ranking metric	Macro-F1
Test-set evaluation	Top 10 CV-ranked pipelines refitted on full training partition

Repeated stratified five-fold cross-validation with three repetitions was applied to the training partition, yielding 15 validation estimates for each pipeline configuration.

S2. Feature Selection Hyperparameters

Three filter-based feature-selection methods were evaluated:

- Variance Threshold (VT)
- ANOVA F-test
- Chi-squared (Chi2)

The number of retained features k was fixed as:

$$k = \min(30, N_f)$$

where N_f represents the number of available input features after preprocessing. Thus, a maximum of 30 features was retained when sufficient features were available.

Feature Selection Method	Implementation	Hyperparameter Setting
--------------------------	----------------	------------------------

Hyperparameter Configuration

Variance Threshold (VT)	VarianceThreshold	threshold = 0.0
ANOVA	SelectKBest(f_classif)	k = min(30, number of features)
Chi-squared	SelectKBest(chi2)	k = min(30, number of features)

For Chi-squared selection, Min-Max normalization was applied because the method requires non-negative feature values. Standard scaling was applied for the remaining feature-selection branches.

S3. Feature Extraction Hyperparameters

Four dimensionality-reduction methods were evaluated:

- Principal Component Analysis (PCA)
- Linear Discriminant Analysis (LDA)
- Independent Component Analysis (ICA)
- Truncated Singular Value Decomposition (SVD)

The number of extracted components was fixed as:

$$n_{components} = \min(20, \max(2, k))$$

where k is the number of selected features.

Feature Extraction Method	Hyperparameters
PCA	n_components = min(20, max(2,k)); random_state = 42
LDA	n_components = min(n_components, number of classes - 1)
ICA	n_components = min(20, max(2,k)); max_iter = 1000; tolerance = 0.01; random_state = 42
Truncated SVD	n_components = min(20, max(2,k)); random_state = 42

The LDA component number was automatically constrained because the maximum number of discriminant dimensions cannot exceed the number of classes minus one.

S4. Classical Classifier Hyperparameters

12 classical machine-learning classifiers were evaluated using fixed configurations.

Classifier	Explicit notebook configuration
DT	DecisionTreeClassifier(random_state=42)
LR	LogisticRegression(max_iter=2000, random_state=42)
KNN	KNeighborsClassifier()
NB	GaussianNB()
SVM	SVM(kernel="rbf", probability=False, random_state=42)
RF	RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
ET	ExtraTreesClassifier(n_estimators=100, random_state=42, n_jobs=-1)
GB	GradientBoostingClassifier(random_state=42)
XGB	XGBClassifier(n_estimators=100, random_state=42, n_jobs=-1, eval_metric="mlogloss", tree_method="hist")

Hyperparameter Configuration

LGBM	LGBMClassifier(n_estimators=100, random_state=42, n_jobs=-1)
CAT	CatBoostClassifier(iterations=100, random_seed=42, verbose=0, loss_function=...)
STACK	RF + XGB + LGBM → Logistic Regression

These configurations were kept unchanged across all FS–FE combinations to avoid introducing classifier-specific tuning bias.

S5. Stacking configuration

Stacking parameter	Value
Base learners	RF, XGB, LGBM
Trees/iterations in each base model	50
Meta learner	Logistic Regression
Meta-LR max iterations	2000
Internal stacking CV	5 folds
passthrough	False
stack_method	"auto"
Parallelization	n_jobs=-1 where supported
Seed	42

For the stacking classifier, Random Forest, XGBoost, and LightGBM were used as base learners and Logistic Regression as the final estimator. Meta-level features were generated using five-fold internal cross-validation ($cv=5$), with `passthrough=False` and automatic selection of the stacking prediction method. The entire stacking model was itself evaluated within the repeated five-fold, three-repeat outer cross-validation protocol.

S6. Pipeline Evaluation Configuration

For each dataset, the main grid is:

$$3 \text{ FS} \times 4 \text{ FE} \times 12 \text{ classifiers} = 144$$

pipeline configurations.

Each undergoes:

$$5 \times 3 = 15$$

outer validation fits.

Thus the main benchmark generates up to:

$$144 \times 15 = 2160$$

outer-fold evaluations **per dataset**, excluding the additional internal fits required by stacking.

S7. Evaluation Metrics

The following metrics were computed:

Metric	Configuration
--------	---------------

Hyperparameter Configuration

Accuracy	Standard accuracy
Macro-F1	Primary ranking metric
Weighted-F1	Imbalance-aware performance
Macro Precision	Class-balanced precision
Macro Recall	Class-balanced recall

The final ranking of pipelines was performed according to Macro-F1, followed by accuracy as a secondary criterion.